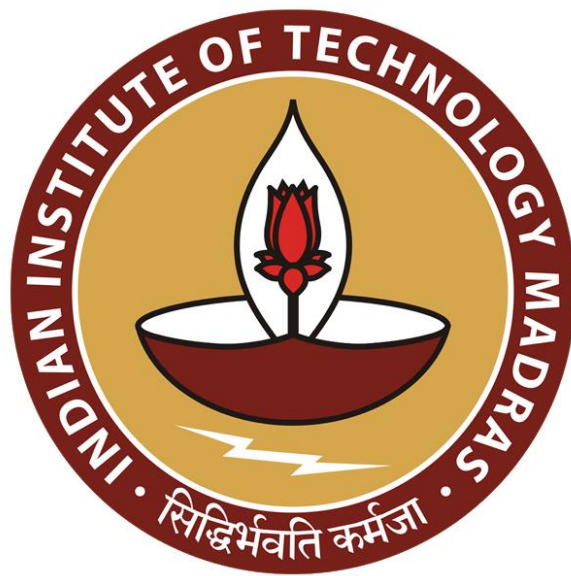




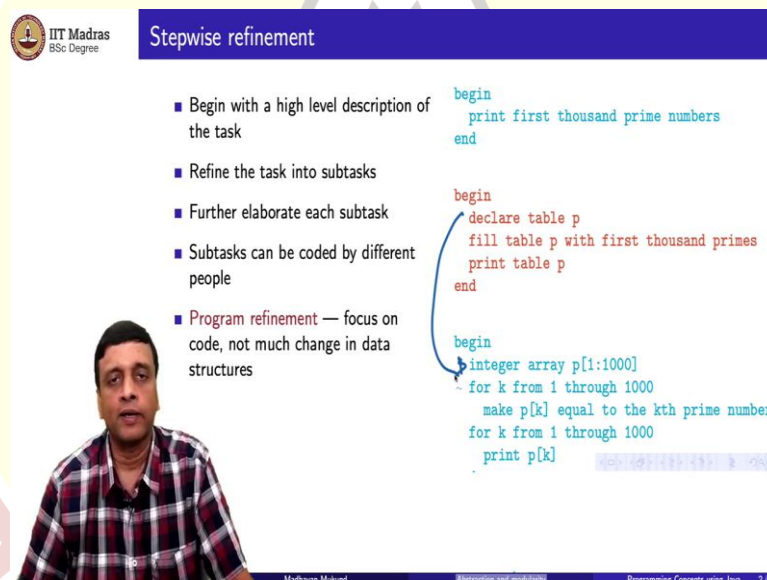
IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Lecture 4
Abstraction and Modularity

So, writing software is a complex process we have to typically build large systems and the only way to do it reliably is to be able to abstract away from the details and build them in parts that is in a modular fashion. So, let us look at abstraction and modularity and how we can embed them into a programming language.

(Refer Slide Time: 00:35)



The slide is titled "Stepwise refinement" and features a list of four steps on the left and three levels of code refinement on the right. A blue arrow points from the first level of code to the second, and another blue arrow points from the second to the third, illustrating the process of refinement.

- Begin with a high level description of the task
- Refine the task into subtasks
- Further elaborate each subtask
- Subtasks can be coded by different people
- Program refinement — focus on code, not much change in data structures

```
begin
  print first thousand prime numbers
end
```

```
begin
  declare table p
  fill table p with first thousand primes
  print table p
end
```

```
begin
  integer array p[1:1000]
  for k from 1 through 1000
    make p[k] equal to the kth prime number
  for k from 1 through 1000
    print p[k]
end
```

So, first let us look at the process of refining a problem down to smaller pieces. So, usually when we have a complex problem we start with a high level description of the task. So, here it says print the first 1000 prime numbers. So, this is the task at hand. So, it does not say anything about how to do it, it just tells us what is to be done. So, as a first step we might decide that we need to store these prime numbers somewhere. So, we need a table.

So, we declare a table p then we fill this table with the first 1000 primes and then we print the table. So, this is slightly more detailed than the first one. So, we have now the sub task which consists of creating some storage filling that storage and printing what is

stored in that table then we could go one more step for instance and elaborate this. So, we can declare that this table is actually an array of 1000 elements and these are all integers.

Then we can write an explicit loop which tells us how we will populate this although this is still abstract because we still have to figure out how to calculate here the kth prime number right. So, it is not quite a solution but it is more concrete it tells us that filling this table can be achieved through a loop which fills it one by one. Similarly printing the table can be achieved by a loop which prints it one by one.

So, at each step we make the functions or the steps the tasks that we want to perform more and more concrete until we get down to something which we can actually run on our system. One advantage of doing it this way is that we are breaking up our task into smaller units which can then be handed out. So, this is a very simple problem of course but if you had a large project in which you had to implement many different functionalities.

You could have a team of software developers who are asked to perform different parts of this development and then put it together to form the whole. So, you break down the overall task like a banking application might it consists of having one team which develops a login screen another team which develops the database to store the data that is going to be created and maintained another team which has some forms to fill in and so on.

And each of these if they are broken down separately can be executed by separate teams to make the task more manageable and achievable. So, this is what is called program refinement in the literature. So, the main thing about this is that we really in the process of program refinement we may make decisions like for instance, if we come from here and we now decide at this point that we need an integer array.

It is unlikely that going forward that integer array is going to have to be changed into something else. So, the data structures as they get created will remain as they are in further steps of the refinement and the main focus is on breaking down the execution that

is the code itself into simpler and simpler steps until we have something that is easy to program.

(Refer Slide Time: 03:30)

IIT Madras
BSc Degree

Data refinement

- Banking application
 - Typical functions: `CreateAccount()`, `Deposit()/Withdraw()`, `PrintStatement()`
 - How do we represent each account?
 - Only need the current balance
 - Overall, an array of balances
 - Refine `PrintStatement()` to include `PrintTransactions()`
 - Now we need to record transactions for each account
 - Data representation also changes
 - Cascading impact on other functions that operate on accounts

Mathavan Muralidharan Abstraction and Modularity Programming Concepts using Java 3 / 8

But sometimes we may need to have what is called data refinement. So, let us consider that we have a banking application and we have functions like creating an account depositing and withdrawing money from one account to another and printing up an account statement. So, to begin with we might see that given the functionality that we ask for the only thing we need to know about an account is how much money is there.

So, if we create an account and we remember the current balance then when we deposit money into it we increment that balance when we withdraw money we decrement the balance. If we transfer its like withdrawing from one account and adding to another account and so on. And when we print a statement for instance we will just print out the current state of the account.

So, this way we could have one value one number it could be an integer or could be a float if we want to remember up to decimals the value of the current balance and the accounts together for all the customers in the bank can just be represented as a simple array. Now the process of refining supposing this printing a statement also involves printing out all the transactions that the customer has done on that account.

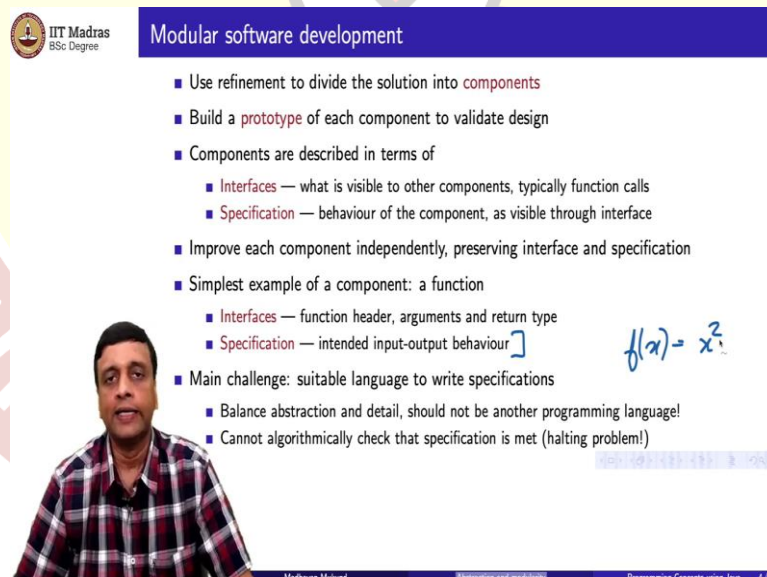
Now we find that the data that we stored earlier the decision we made about having only the current balance representing the value of the account is no longer sufficient. So, we

need to change the representation of each account to include a history of all the transactions and then perhaps we do not even need the current balance because if we have we start from 0 and we have an initial deposit we can actually compute the current balance by just running to the transactions.

So, in more realistic situations program refinement can also have a cascading impact on the data structures that we are using and now remember that if we change the data structure for print statement it will also affect how deposit and withdraw work. Because now earlier deposit and withdraw for instance were just updating the balance but now instead they have to record the details of the transaction in the transaction log that we are associating each account.

So, the change in one step may affect other steps and this is a problem which arises because we have data refinement. So, we have data in our program which is shared across these different modules and this data is getting updated because one of the modules when it gets refined requires a change in the data structure.

(Refer Slide Time: 05:56)



IIT Madras
BSc Degree

Modular software development

- Use refinement to divide the solution into **components**
- Build a **prototype** of each component to validate design
- Components are described in terms of
 - **Interfaces** — what is visible to other components, typically function calls
 - **Specification** — behaviour of the component, as visible through interface
- Improve each component independently, preserving interface and specification
- Simplest example of a component: a function
 - **Interfaces** — function header, arguments and return type
 - **Specification** — intended input-output behaviour
- Main challenge: suitable language to write specifications
 - Balance abstraction and detail, should not be another programming language!
 - Cannot algorithmically check that specification is met (halting problem!)

$f(x) = x^2$

Madhavan Mukund Abstraction and modularity Programming Concepts using Java 4 / 6

So, these are all broad guidelines. So, in general what our aim is when we are building complex software is to use refinement to divide the solution into components. Now the advantage of building these defining these components in a suitably modular way is that we can then start with what is called a prototype. A prototype is like a simple implementation of each component.

And then we can check that this simple implementation fits together to achieve the task at hand. So, it is in fact going to serve the purpose the simple implementation is not our final one it may not be fast enough it may not scale up in terms of the size that we want and so on but it will at least justify that all the functionality that we expect from our system is correctly captured through our design.

So, in order to build this prototype in order to make sure that these components interact properly and then can be developed separately we need to specify each component in terms of two parts one is the interface. The interface tells us how this component interacts with the other components in terms of what functions you can call on this component from outside and what parts of this function are going to be visible either through these function calls or otherwise.

So, what data from this component can be made visible and what functions can be invoked. The specification on the other hand is a behavioral system. It tells me what this component is supposed to do. So, for example if my component one of my components is to withdraw money from an account then I say that if I pass a number an amount and an account number it must decrement that amount from that account number.

So, that is what a withdraw thing should do. So, the specification is a behavioural requirement the interface is a static structural requirement which tells me which functions are legal to call on this and they behave the specification will tell me what these functions are supposed to do. If we have this kind of a design. So, we have these different components right and we know exactly what we can ask of each component then if I go inside and if I change this right if I make a component better.

So, imagine for instance that you have to sort something. So, you write a sorting algorithm which initially you just write a very simple sorting algorithm which is inefficient. Now tomorrow if you replace that sorting algorithm by an efficient sorting algorithm from the external perspective it still takes an array or a list and returns a sorted list. So, this is what we mean by saying that each component we can improve make it more efficient make it more scalable.

Independent of the other things while preserving this interface and specification. So, it does not affect the way in which other components that interact with this have to behave. So, unlike the previous case where we saw that if you had to refine the print statement module and it had a cascade impact to others we should reach a level of separation of concerns where this does not happen.

So, in normally when we are dealing with code the simplest unit which expresses this notion of a component is a function. So, when we write a function we are encapsulating some repeated operation that we perform on data. For instance we might repeatedly be taking x times x . Now we want to say ok instead of x times x let me just write a function square with squares x . So, that I know that this is what is happening.

So, the interface in this case will be the name of the function and the signature as it is called which is the order of these arguments and what types these arguments have and what value or what type of value the function will return. So, this function header as it is called is the interface and the specification is the intended input output behaviour. So, if our function is to square then it says given a number it will return the square of that number.

So, of course the main challenge which this is actually the second step it is easy to write the interface because that is just syntax in the programming language that you are using but the specification is something more complicated. If it is a numeric thing I can describe it mathematically I can say something like f of x is equal to x squared. But if it is something complicated like update some entries in a table then it is very difficult to write it down in a very precise way without actually procedurally saying go to this entry go to that entry and so on.

So, in other words if you really write down a detailed specification of a complex program it might actually look like another program. So, this is the danger the specification language could be as complicated or as verbose as a programming language and it if it means that to write down the specification you effective to write a program then you are kind of going in circles because we want the specification to help us in writing a program.

So, this is a tricky thing and it is not something easy to solve there are many just like there are many programming languages there are many specification languages also. So, this although informally this is what we want to do very often we have to manage with some kind of informal requirements and informal specifications because we just do not have that kind of rigor in our specification language to write down everything that we want.

The other thing which makes this whole process a little difficult is that there is no actual way to validate that a specification is met by the code that you write. Remember that we mentioned earlier that one reason to use type checking is to try and avoid errors early because we really do not have a way to check that a program is correct. So, this halting problem of turing machines which is a bottleneck for program verification is also the same bottleneck applies to program specific.

Because program verification essentially is checking whether program behaves correctly on its input or not in this case we even have a formal description of what the program should do but still this matching is not possible automatically. So, while we would like to do this modular software development in a formal way this part of you know actually building the components and then validating them often results is resolved by using some kind of a manual approximation of this because there is no automatic way to do it.

But nevertheless this kind of refinement based way of building large systems is the only logical way to do it because you just cannot start with enormous like supposing somebody tells you organize an event. So, we are going to have an event next week where we are going to call all the alumni in our college organize it. Now at that level it is very difficult to imagine what you need to do.

So, you need to start thinking okay organizing event means what I have to arrange for instance a place where the event will be held. So, I need to think of an auditorium or a hall or a ground or something I need to know how many people are likely to come I need to know whether I need chairs or not I need to know whether it is going to be food or not. How many people are going to come how long is the event do I need lighting do I need sound.

So, all these things become part of this refinement of the task into sub tasks then you can start delegating you can ask somebody to go and arrange the furniture you can ask somebody else to make arrangement for the lighting and so on.

(Refer Slide Time: 12:46)



Summary

- Solving a complex task requires breaking it down into manageable components
 - Top down: refine the task into subtasks
 - Bottom up: combine simple building blocks
- Modular description of components
 - Interface and specification
 - Build prototype implementation to validate design
 - Reimplement the components independently, preserving interface and specification
- PL support for abstraction
 - Control flow: functions and procedures
 - Data: Abstract data types, object-oriented programming



Madhavan Malur

Abstraction and modularity

Programming Concepts using Java

6/6

So, now what we want at our level is that programming languages should have some support for this kind of abstraction where we can take modules and push them into a black box. So, that we can then combine these black boxes at a higher level and reason about our code rather than raising with every line in output. So, for control that is for the execution part of it the normal abstraction as we said is a function or a procedure.

So, it encapsulates a block of code which I can reuse in different context by passing different parameters and so on. So, it does the same thing I do it in a number of different context and depending on what parameters I pass I know what to expect. The other part of it which is what will be largely the focus of this course is what is called data abstraction. So, we have seen before the notion of an abstract data type in the data structures course.

So, abstract data type tells us at a high level how some values are organized and what functions are allowed them. For example a priority queue is an abstract data type right it is a collection of values with priorities and you want to be able to delete the highest or remove the one with the highest priority and we need to be able to insert a new element with arbitrary priority and do all this efficiently.

Now whether we implement that priority queue as a sorted list or as a heap or so on is something which is independent of this external functionality. So, the set of values and the operations defined on them define the interface. So, to speak and the specification of the abstract data type and the internal representation ideally should not be accessible at all I should not be aware of whether you are using a sorted list or a heap for a priority queue.

So, that I do not make use of that fact in my code. So, that tomorrow if that internal representation changes it should not make my code invalid. So, the interaction with an abstract data type should be restricted to what is called the public interface. So, for example if you have a stack as a simplest example, so, usually we would put a stack as a list or an array and we will have push and pop at one end.

So, this is the legal operation. So, if I look at the abstract data type for stack it will have two operations and only two operations to manipulate that list which is to add an element at the end of the list and to remove the last element in the list. On the other hand if I know that it is a stack I mean if I know it is an array or a list I can actually go inside and start extracting values from the middle of the list or even updating them.

And this is against the specification of a stack and this should not be allowed. So, this is what we mean by saying that the private implementation should not be accessible outside the stack. So, we can take this abstraction of an abstract data type and make it even richer. So, we can organize it in a hierarchy and in this hierarchy then we can say that this abstract data type is a special case of another abstract data type.

So, I can inherit certain things and add some functionality. So, this is what is called sub typing and inheritance. So, this is really the foundation of object oriented programming. So, object oriented programming essentially starts with the notion of abstract data types. So, in an abstract data type we have storage we have functions on the storage which are defined along with the storage.

So, you can tell a function a stack to pop itself you can tell a stack I want to push a value on to you right and then you can then have a further level of organization where we say that this abstract data type is a special case of that abstract data type. So, we have a

hierarchy. So, this is what we will see when we do object oriented programming. So, to summarize if you want to solve any complex task it does not matter whether it is programming or otherwise we need to break it down into manageable components.

So, there are two broad ways to do this. So, one which we looked at is this refinement we start with the high level thing and we break it down into sub tasks. Also we could have a library of tasks which are already available to us right and then we can say whether we can combine them in order to form the task. So, we might already know that we have a good sorting routine.

So, if I need to sort something I do not have to re-implement the sort I can just pick up that building block that I have. So, we can also do some bottom up construction with sub tasks which are already available to us. So, ideally we need a modular description of the parts that make up our whole. So, we call these components and each part is defined in terms of its interface which is how it interacts with the other components. So, this is what we roughly call the public part of it.

The functions that you can call the values that are available and the behavioral specification what these functions do. So, if I pass you a certain piece of data what should I expect in return? So, once we have these segregated components which are completely defined in terms of the interface and implementation I can then develop each component in isolation.

So, first I can build a simple prototype where each component is built in a minimal fashion and then check that the design is correct and once this is done I can gradually build up refined versions of each pro each component. So, that I get a more functional and effective system. So, to do all this we need some amount of support in our programming language.

So, in almost every programming language will have an ability to define functions and procedures to encapsulate bodies of code that we need repeatedly. And the other part which will play an important role in what we are going to do in this course is data abstraction. So, we would like to define in a clean way abstract data types and combining

combine them together in a hierarchy and this is what leads us to object oriented programming.

